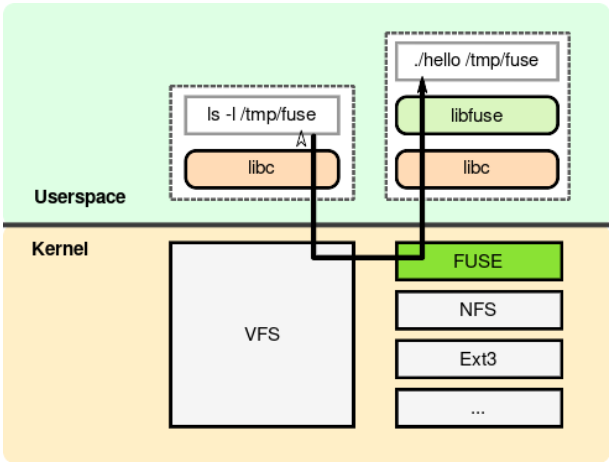
# TP No2: Création d'un système de fichier

November 25, 2025




Ce second TP consiste à regarder les sources d'un système de fichiers très simple, créé pour l'occasion, appelé **SFS (Simple FileSystem)**.

SFS est un système de fichiers basé sur une notion simplifiée **d'inode**. Les volumes SFS sont en fait simplement stockées dans des fichiers standards.

Un volume SFS peut donc être vu comme une archive contenant une arborescence de fichiers / répertoires.

L'objectif est de rajouter les fonctionnalités nécessaire pour faire fonctionner ce système de fichiers.

SFS s'appuie sur sur **FUSE (File System User Space)** qui permet de fabriquer un système de fichier utilisable en espace utilisateur grâce à un module qui communique avec la bibliothèque **libfuse**.

# 1 SFS Prise en main

## 1.1 Installation

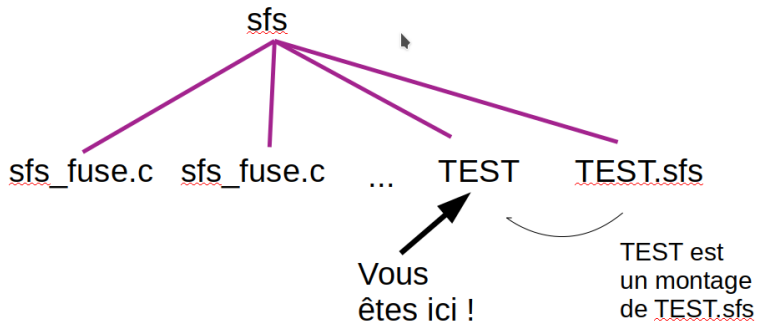
1. cd /tmp pour les postes de l'université
2. chargez l'archive sfs.tar.
3. tar -xf sfs.tar
4. cd ./sfs
5. ls -1

## 1.2 Execution

Vous trouverez dans le répertoire un script **test-fuse.fs**.

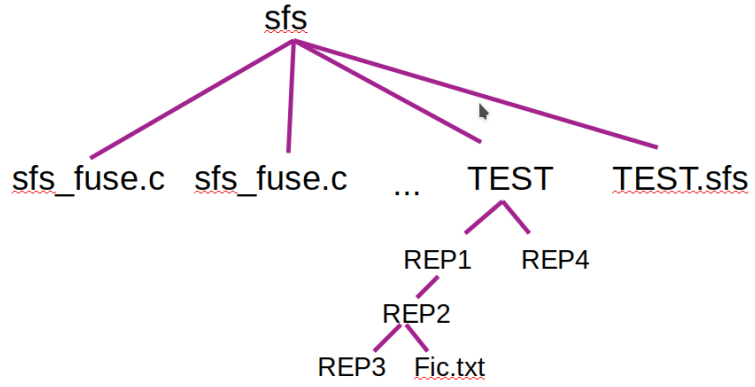
Ce script va :

1. compiler les fichiers c
2. créer plusieurs commande sfs_fuse, sfs_shell
3. créer un fichier TEST.sfs
4. créer un système de fichier sfs dans TEST.sfs
5. monter ce fichier sous TEST
6. ouvrir un shell dans le répertoire




## 1.3 manipulation

Créez l'arborescence suivante et affichez les no d'inode avec la commande tree:



```
$mkdir -p REP1/REP2/REP3 REP4
$echo "TEST SIMPLE" > ./REP1/REP2/fic1.txt
$tree --inodes .
```

**Question 1**

Donnez le no d'inode du répertoire TEST. Expliquez ce que veux dire monter un système de fichier.

Pour démonter le système de fichier tapez exit.

### 1.4 Manipulation directe des inodes / blocs

Si vous sortez du shell avec la commande exit, votre système de fichier sera démonté. Si vous effectuez un ls ./TEST vous n'aurez plus de trace de vos répertoires.

Pour manipuler directement les inodes et les blocs de votre système de fichier vous allez utiliser la commande sfs_shell.

```
rlwrap ./sfs_shell TEST.sfs
```

Les premières commandes que vous utiliserez sont

- **show_inode** affiche les informations d'un inode
- **dump_entries** affiche le contenu d'un bloc de fichier de type répertoire
- **dump_data** affiche le contenu d'un bloc de données au format ASCII

Pour modifier le contenu des données pointées par un inode vous utiliserez :

- **write** écris une chaîne à l'offset dans les blocs de données.

**Question 2**

Ajoutez du texte au fichier fic1.txt

**Question 3**

Créez le fichier fic2.txt dans REP4 et ajoutez lui du texte..

## 2 Modification du code de SFS

Le but de cette partie **est de modifier le code de sfs** pour ajouter de nouvelles fonctionnalités. Avant cela il est important de bien comprendre les différentes structures qui représente le système de fichier.

### 2.1 Les inodes

SFS est basé sur le type d'inodes suivant contenu dans le fichier **sfs_volume.h**:

```
typedef struct inode {
      int type;                          // inode type: S_IFREG, S_IFDIR
         , S_IFLNK
      int size;                          // size of the corresponding
         file in bytes
      int nlinks;                        // number of hard links to this
          inode (0 means the inode is free)
      int blocks[NB_DIRECT_BLOCKS+2];    // blocks for the data (last 2
         blocks for single / double indirection blocks)
  } inode_t;
```

Les constantes

1. S_IFREG fichier régulier
2. S_IFDIR répertoire
3. S_IFLNK lien symbolic
4. NB_DIRECT_BLOCKS blocs de données en accès directe

Chaque inode est représenté par son numéro (type int), avec les constantes suivantes :

1. l'inode 0 n'est pas utilisé,
2. l'inode 1 BADBLOCKS_INODE pourrait servir à référencer les blocs défectueux,
3. l'inode 2 ROOT_INODE est l'inode de la racine de la partition.

## 2.2 Blocs de données

```
// size (in bytes) of blocks
#define BLOCK_SIZE 512

// type synonym for "raw" bytes
typedef unsigned char byte;

// type synonym for blocks
typedef byte block_t[BLOCK_SIZE];
```

## 2.3 Indexation des inodes / blocs

Les blocs et les inodes sont représentés par des entiers (positifs).

Les fonctions **sfs_get_inode** / **sfs_get_block** prennent un argument entier et renvoient un pointeur vers l'inode / bloc correspondant.

Attention, ces fonctions ne font aucune vérification sur l'entier qui leur est fourni. Si vous les utilisez avec des entiers trop grands (ou négatifs), le résultat n'est pas spécifié et votre programme risque de provoquer une erreur de segmentation.

## 2.4 Obtenir les statistiques du système de fichier

L'appel système **statvfs** permet d'obtenir des informations sur un système de fichiers :

1. taille des blocs
2. taille du système de fichiers
3. nombre de blocs libres

La fonction **sfs_statvfs** est définie dans **sfs_posix.c**, elle contient des appels aux fonctions:

- sfs_count_free_blocks
- sfs_count_free_inodes

**Exercice 1**

**Modifiez le fichier sfs_low_level.c qui contient ces fonctions pour afficher les statistiques du système de fichier soit avec la commande df -i sur le sf monté soit avec le commande info dans le sfs_shell.**

Les éléments suivants sont nécessaires:

1. **DATA_SIZE**: taille de la partition, en nombre de blocs (fichier sfs_parameters.h)
2. **sfs_block_in_use** pour vérifier dans la bitmap si un numéro de bloc correspond à un bloc utilisé
3. **ROOT_INODE** : premier numéro d'inode utilisable (pour la racine)
4. **NB_INODES** : nombre total d'inodes sur la partition
5. **sfs_get_inode** : pour récupérer un inode à partir de son numéro.

## 2.5 Ajout des liens physiques

Un lien dans un système de fichier est une référence supplémentaire à un fichier qui permet de de le manipuler depuis un autre répertoire ou à l'exécuter si c'est une commande sans avoir à la copier par example. Il existe deux types de liens les liens physiques et les liens logiques.

Pour créer un lien et tester son fonctionnement :

```
$touch fic
$ln ./fic ./lnfic
$ls -li

$echo "test" > ./lnfic
$cat ./fic
$stat ./lnfic

$rm ./fic
$stat ./lnfic
```

**Question 4**

Expliquez la façon dont les liens physiques sont créés. Commentez la valeur de Lien dans la commande stat.

**Exercice 2**

Modifiez les fonctions **sfs_unlink_inode** et sfs_link_inode du fichier **sfs_low_level.c**. Pour implémenter la gestion des liens physiques.

## 2.6 Ajout des liens logiques

Effectuez les commandes suivantes

```
$touch fic2
$ln -s ./fic2 ./lsfic2
$ls -li

$echo "test" > ./lsfic2
$cat ./fic2

$mv ./fic2 autrenom
$ls -l

$mv ./autrenom fic2
$ls -l
```

**Question 5**

Pouvez-vous émettre une hypothèse sur le fonctionnement du lien logique ?

**Exercice 3**

Ajoutez la gestion des liens symboliques dans SFS, il faut compléter les deux fonctions (dans le fichier sfs_low_level.c) suivantes :

- **sfs_writelink_inode** qui permet d'initialiser un inode au type S_IFLNK et d'inscrire la cible du lien au bon endroit.
- **sfs_readlink_inode** qui récupère la cible d'un lien symbolique.

Dans le code fournit, ces deux fonctions récupèrent l'inode demandé et renvoient une erreur s'il ne s'agit pas d'un lien symbolique. Elle ne stockent, ni ne renvoie la cible du lien symbolique.

Dans un premier temps, **stockez la cible du lien symbolique directement dans le tableau nd->blocks** de l'inode en utilisant **strncpy**. Vous pourrez stocker la taille de la chaine cible dans le champs size de l'inode.

La stratégie précédente ne fonctionne que si le nom de la cible est suffisamment petite pour tenir sur les quelques dizaines d'octets du tableau nd->nb_blocks.

**Si la cible est trop grande, utilisez la fonction sfs_write_to_block pour stocker la**

**cible dans un nouveau bloc de données associé à l'inode. Vous pourrez également stocker la taille de la chaine cible dans le champs size de l'inode.**

Taille des modifications (total) : environ une dizaine de lignes très similaires pour chacune des fonctions.

## 2.7 Blocs à accès indirect

Actuellement, les blocs de données associés à un inode dans SFS sont tous à accès direct. Le tableau blocks d'un inode contient **NB_DIRECT_BLOCKS** cases référençant des blocs de données.

La taille maximale d'un fichier est donc égale a **NB_DIRECT_BLOCKS * BLOCK_SIZE**, soit quelques kibioctets...

**Pour augmenter légèrement la taille possible des fichiers, nous allons ajouter la gestion de blocs indirects. Le tableau blocks d'un inode contient une case supplémentaire, actuellement non utilisée.**

Lorsque cela est nécessaire, cette case sera utilisée pour référencer un bloc contenant **BLOCK_SIZE / sizeof(int)** blocs de données.

**Exercice 4**

Ajoutez la gestion des blocs indirects pour les fichiers dans SFS.

- **La fonction sfs_read**(sfs_posix.c) est programmée à partir de la fonction sfs_read_from_inode (sfs_low_level.c), elle même programmée à partir de sfs_read_from_block (sfs_low_level.c), qui utilise sfs_get_data_block_nb_from_inode pour chercher les numéros de blocs à lire.
- **La fonction sfs_write**(sfs_posix.c) est programmée à partir de la fonction sfs_write_to_inode (sfs_low_level.c), elle même programmée à partir de :sfs_write_to_block (sfs_low_level.c).

Taille des modifications :

- sfs_get_data_block_nb_from_inode, quelques lignes
- sfs_write_to_block, une quinzaine de lignes

N'oubliez pas de documenter vos tests pour mettre en ouvre cette gestion des blocs indirects.

## 2.8 Fichiers creux

Les systèmes avec inodes permettent de créer des fichiers creux : il suffit de mettre un numéro de bloc spécial qui n'est pas alloué sur le support. Les opérations de lecture sur un tel bloc renvoient des 0, et les opérations d'écriture vont provoquer l'allocation d'un vrai bloc...

Vous pouvez créer un fichier creux avec la commande :

```
dd if=/dev/zero of=sparse_file bs=1 count=0 seek=512M
```

Pour SFS, nous allons utiliser le numéro de bloc ZERO_BLOCK (constante définie à 0) pour représenter un bloc de 0 non alloué... (Plus précisément, ce bloc est alloué mais en "lecture seule".

Ajoutez la gestion des fichiers creux dans SFS.

Comme précédemment, les fonctions POSIX qui accèdent au contenu du fichier sont read et write (ainsi que truncate). Comme le bloc ZERO_BLOCK est un vrai bloc ne contenant que des 0, il n'est pas nécessaire de modifier les fonctions de lecture. Il reste donc sfs_truncate_inode, qui accède aux blocs à travers les fonctions

allocate_datablocks qui doit toujours allouer le bloc ZERO_BLOCK comme nouveau bloc,
deallocate_datablocks qui doit libérer les blocs standards, mais ne rien libérer lorsqu'il s'agit d'un bloc de zéros..

Pour l'appel système write, il faudra modifier la fonction sfs_write_to_block. C'est notamment cette fonction qui, lors de l'écriture dans un blocs de zéros, le remplacera par un bloc standard, cherché avec la fonction sfs_find_free_block.

Il faut également modifier la fonction sfs_count_used_blocks_inode qui permet de calculer la taille occupée par les données d'un inode.

Taille des modifications :

1. allocate_datablocks, une ligne,
2. deallocate_datablocks, quelques lignes,
3. sfs_write_to_block, une dizaine de lignes,
4. sfs_count_used_blocks_inode, quelques lignes.